COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-CPE21S3 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	8/4/2026	**DATE SUBMITTED:**	8/6/2026
NAME:	Gabriel Immanuel R. Serrano	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 5.1 Queues

1. PROCEDURE OUTPUT

ILO A

ILOA > main.cpp > ...

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main(){
    //create an object
    std::queue<char> myQ;

    myQ.push('G');
    myQ.push('A');
    myQ.push('B');
    display(myQ);
    //try all member function
    //empty - bool either 0 or 1
     std::cout<<"The list is: ";
     if(myQ.empty()){
        std::cout << "Empty";
     }else{
        std::cout << "Not Empty";
     }
    //size
    std::cout<<"\nThe size is: " <<myQ.size() << std::endl;
    //front
    std::cout<<"The front is: " << myQ.front() << std::endl;
    //back
    std::cout<<"The back is: " << myQ.back() << std::endl;
    //pop
    std::cout<<"Popping the front!" << std::endl;
    myQ.pop();
```

```
31      //display
32      std::cout<<"After the popping:";
33      display(myQ);
34      //push
35      std::cout<<"Pushing the letter:";
36      myQ.push('N');
37      //display
38      display(myQ);
39
40      return 0;
41  }
42
43  void display(std::queue<char> copyQ){
44      //create a copy of the queue that was passed
45      std::queue<char> c = copyQ;
46      while(!c.empty()){
47          std::cout<<" " << c.front();
48          c.pop();
49      }
50      std::cout<<"\n";
51  }
```

Output

```
[Running] cd "c:\Users\TIPQC\Documen
G A B
The list is: Not Empty
The size is: 3
The front is: G
The back is: B
Popping the front!
After the popping: A B
Pushing the letter: A B N
```

Observation:

In this code we implemented a STL library for queue. The queue is almost the same as the linked list, it checks the head and the back, but instead of last in first out, it is first in first out. We created a character queue object that pushed the letters G,A,B into it, and then we explored all the functions by checking whether it was empty, its size, front, back, and pop operations. But it was lacking of displayall function so we created it by using a copy queue to save the original queue while displaying all the characters inside the queue.

ILO B

Main.py

main.cpp ILOB | queueArr.h | main.cpp ILOA | queueLL.h

ILOB > main.cpp > ...

```cpp
#include <iostream>
#include "queueLL.h"


int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('G', &front, &back);
    enqueue('G', &front, &back);
    enqueue('A', &front, &back);
    enqueue('B', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;

    dequeue(&front, &back);
    std::cout << front->data << " " << back->data << std::endl;

    std::cout << "The front is: " << std::endl;
    checkFront(&front);
    std::cout << "The back is: " << std::endl;
    checkBack(&back);
    std::cout << "Display All: " << std::endl;
    displayAll(&front, &back);

    return 0;
}
```

queueLL.h

ILOB > C queueLL.h > ...

```h
#ifndef QUEUELL_H
#define QUEUELL_H

//create a class the node
template <typename T>
class qNode{
    public:
        T data;
        qNode<T> *next;
};

//creates the new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

//enqueue operation
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    //create the new node
    qNode<T>* newNode = new_node(newData);

    //inserting into an empty queue
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }
    //inserting an item into a non empty queue
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}
```

```
36
37  //dequeue
38  template <typename T>
39  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
40      //create a temporary node
41      qNode<T>* deleteNode = nullptr; // temp variable
42      deleteNode = (*frontPtr);
43
44      //check if the queue has only 1 node
45      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
46          (*frontPtr) = nullptr;
47          (*backPtr) = nullptr;
48          delete deleteNode;
49          return;
50      }
51      //delete the node
52      (*frontPtr) = (*frontPtr)->next;
53      deleteNode->next = nullptr;
54      delete deleteNode;
55  }
56
57  // check the front
58  template <typename T>
59  void checkFront(qNode<T>** frontPtr){
60      if((*frontPtr) != nullptr){
61          qNode<T>* front = (*frontPtr);
62          std::cout << front->data << std::endl;
63          return;
64      }
65  }
```

```
66  //check the back
67  template <typename T>
68  void checkBack(qNode<T>** backPtr){
69      if((*backPtr) != nullptr){
70          qNode<T>* back = (*backPtr);
71          std::cout << back->data << std::endl;
72          return;
73      }
74  }
75
76  //display all
77  template <typename T>
78  void displayAll(qNode<T>** frontPtr, qNode<T>** backPtr){
79      qNode<T>* c = (*frontPtr);
80      qNode<T>* b = (*backPtr);
81
82      if( c == nullptr && b == nullptr){
83          std::cout << "The queue is empty!";
84          return;
85      }
86
87      while(c != nullptr){
88          std::cout << c->data;
89          c = c->next;
90      }
91
92  }
93
94  #endif
```

Output:

```
G B
G B
The front is:
G
The back is:
B
Display All:
GAB
```

Observation :

In this activity , instead of using the STL queue library, we created our own queue implementation using a header file. Inside the header file, we defined a Node class and implemented the queue operations, such as enqueue for inserting data, dequeue for removing the front node, checkFront for checking the first element, checkBack for checking the last element, and displayAll for displaying all the elements in the queue. The implementation is very similar to the STL queue library, but we created our own version that allows us to modify functions based on the specific requirements we need for our program, instead of being limited to the function that the library has to offer.

ILO B 2
Main.cpp

ILOB2 > main.cpp > main()

```cpp
#include <iostream>
#include "queueArr.h"

int main(){
    QueueArr<int> q(5);

    //checking if the queue is empty and its size
    std::cout <<"Is the queue empty: "<<q.isEmpty() << std::endl;
    std::cout <<"The size of the queue is: "<<q.size() << std::endl;

    q.enqueue(6);
    q.enqueue(9);
    q.enqueue(7);

    std::cout <<"\nAfter enqueuing 6, 9 and 7" << std::endl;

    std::cout <<"Is the queue empty: "<<q.isEmpty() << std::endl;
    std::cout <<"The size of the queue is: "<<q.size() << std::endl;
    std::cout <<"The front of the queue is: " << q.front() << std::endl;
    std::cout <<"The back of the queue is: " << q.back() << std::endl;

    std::cout <<"\nDequeuing an element: " << q.dequeue() << std::endl;

    std::cout <<"\nAfter dequeuing an element" << std::endl;
    std::cout <<"Is the queue empty: "<<q.isEmpty() << std::endl;
    std::cout <<"The size of the queue is: "<<q.size() << std::endl;
    std::cout <<"The front of the queue is: " << q.front() << std::endl;
    std::cout <<"The back of the queue is: " << q.back() << std::endl;

    //copy constructor
    QueueArr<int> q2 = q;
    std::cout <<"\nAfter copying the queue" << std::endl;
    std::cout <<"Is the queue empty: "<<q2.isEmpty() << std::endl;
    std::cout <<"The size of the queue is: "<<q2.size() << std::endl;
    std::cout <<"The front of the queue is: " << q2.front() << std::endl;
    std::cout <<"The back of the queue is: " << q2.back() << std::endl;

    //copy assignment operator
    QueueArr<int> q3(9);
```

```
40      q3 = q;
41      std::cout <<"\nAfter assigning the queue" << std::endl;
42      std::cout <<"Is the queue empty: "<<q3.isEmpty() << std::endl;
43      std::cout <<"The size of the queue is: "<<q3.size() << std::endl;
44      std::cout <<"The front of the queue is: " << q3.front() << std::endl;
45      std::cout <<"The back of the queue is: " << q3.back() << std::endl;
46
47
48      //testing the clear
49      q.clear();
50      std::cout <<"\nAfter clearing the queue" << std::endl;
51      std::cout <<"Is the queue empty: "<<q.isEmpty() << std::endl;
52      std::cout <<"The size of the queue is: "<<q.size() << std::endl;
53
54      //testing the queue overflow
55      QueueArr<int> q4(2);
56      std::cout <<"\nOverflowing the Queue" << std::endl;
57      q4.enqueue(1);
58      q4.enqueue(2);
59      q4.enqueue(3);
60
61      return 0;
62  }
```

Header file

ILOB2 > C queueArr.h > ...

```
1   #ifndef QUEUEARR_H
2   #define QUEUEARR_H
3
4   #include <iostream>
5
6   template<typename T>
7   class QueueArr{
8   private:
9       T *q_arr;
10      int q_capacity;
11      int q_size;
12      int q_front;
13      int q_back;
14
```

```
15  public:
16      //constructor
17      QueueArr(int capacity){
18          q_capacity = capacity;
19          q_arr = new T[q_capacity];
20          q_size = 0;
21          q_front = 0;
22          q_back = -1;
23      }
24
25      //copy constructor
26      QueueArr(const QueueArr<T>& other){
27          q_capacity = other.q_capacity;
28          q_size = other.q_size;
29          q_front = other.q_front;
30          q_back = other.q_back;
31          q_arr = new T[q_capacity];
32          for(int i = 0; i < q_capacity; i++){
33              q_arr[i] = other.q_arr[i];
34          }
35      }
36
37      //copy assignment operator
38      QueueArr<T>& operator=(const QueueArr<T>& other){
39          if (this == &other){
40              return *this;
41          }
42          delete[] q_arr;
43          q_capacity = other.q_capacity;
44          q_size = other.q_size;
45          q_front = other.q_front;
46          q_back = other.q_back;
47          q_arr = new T[q_capacity];
48          for(int i = 0; i < q_capacity; i++){
```

```
49              q_arr[i] = other.q_arr[i];
50          }
51          return *this;
52      }
53
54      //Destructor
55      ~QueueArr(){
56          delete[] q_arr;
57      }
58
59      //check if empty
60      bool isEmpty(){
61          return q_size == 0;
62      }
63      //check the size
64      int size(){
65          return q_size;
66      }
67      //clear the queue
68      void clear(){
69          q_size = 0;
70          q_front = 0;
71          q_back = -1;
72      }
73      //check the fornt
74      T front(){
75          if(isEmpty()) return T();
76          return q_arr[q_front];
77      }
```

```
78
79      //check the back
80      T back(){
81          if(isEmpty()) return T();
82          return q_arr[q_back];
83      }
84
85      //enqueue
86      void enqueue(const T& value){
87          if(q_size == q_capacity){
88              std::cout << "Queue Overflow" << std::endl;
89              return;
90          }
91          q_back = (q_back + 1) % q_capacity;
92          q_arr[q_back] = value;
93          q_size++;
94      }
95
96      //dequeue
97      T dequeue(){
98          if(isEmpty()) return T();
99          T value = q_arr[q_front];
100         q_front = (q_front + 1) % q_capacity;
101         q_size--;
102         return value;
103     }
104 };
105
106 #endif
```

OUTPUT

```
Is the queue empty: 1
The size of the queue is: 0

After enqueuing 6, 9 and 7
Is the queue empty: 0
The size of the queue is: 3
The front of the queue is: 6
The back of the queue is: 7

Dequeuing an element: 6

After dequeuing an element
Is the queue empty: 0
The size of the queue is: 2
The front of the queue is: 9
The back of the queue is: 7

After copying the queue
Is the queue empty: 0
The size of the queue is: 2
The front of the queue is: 9
The back of the queue is: 7
```

```
After assigning the queue
Is the queue empty: 0
The size of the queue is: 2
The front of the queue is: 9
The back of the queue is: 7

After clearing the queue
Is the queue empty: 1
The size of the queue is: 0

Overflowing the Queue
Queue Overflow
```

Code Analysis :

For ILO B.2, we created an object named q using the class we created. After creating the object, we first checked whether the queue was empty using the isEmpty() function and checked its current size using the size() function. Since there are no elements inside the queue, it displays the queue as empty and the size to be 0. Next, we inserted three elements into the queue using the enqueue() function. This function adds a new element to the queue. After inserting the elements, we used the front() and back() function to display the first and last element inside the queue. Then we used the dequeue() function to remove the first element, this shows the FIFO on the queue. After we removed the element, we checked again the queue size and displayed the new front and back. We also tested the copy constructor by creating a new queue object with the original content of the first queue. Then we tested the copy assignment operator by assigning the original queue to another queue object. We tested the clear() function to remove all the elements from the queue and reset it to the empty shell. Finally, we tested the queue overflow by creating another queue with a maximum capacity of two and inserting three elements inside of it.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Main.cpp

Supplementary > main.cpp > main()

```cpp
#include <iostream>
#include "suppQueue.h"

int main(){
    Printer myPrinter;

    std::cout << "Adding jobs to the queue..." << std::endl;
    myPrinter.addJob(Job(1, "Dirk Sotto", 5));
    myPrinter.addJob(Job(2, "John Victoria", 10));

    std::cout <<"\nProcessing jobs in the queue..." << std::endl;
    myPrinter.processJob();

    std::cout <<"\n Adding more jobs to the queue..." << std::endl;
    myPrinter.addJob(Job(3, "Alice Volkswagon", 15));
    myPrinter.addJob(Job(4, "King Bob", 20));

    std::cout <<"\nProcessing remaining jobs in the queue..." << std::endl;
    myPrinter.processAllJobs();

    std::cout <<"\nProcessing remaining jobs in the queue..." << std::endl;
    myPrinter.processAllJobs();


    return 0;
}
```

Header file

Supplementary > C suppQueue.h > ...

```h
#ifndef SUPPQUEUE_H
#define SUPPQUEUE_H

#include <iostream>
#include <string>

class Job{
public:
    int jobID;
    std::string idName;
    int pages;

    Job(){
        jobID = 0;
        idName = "";
        pages = 0;
    }
    Job(int i, std::string n, int p){
        jobID = i;
        idName = n;
        pages = p;
    }
};

//create a class the node
template <typename T>
class qNode{
    public:
        T data;
        qNode<T> *next;
};

```

```
33  //creates the new node
34  template <typename T>
35  qNode<T>* new_node(T newData){
36      qNode<T>* newNode = new qNode<T>;
37      newNode->data = newData;
38      newNode->next = nullptr;
39      return newNode;
40  }
41
42  //enqueue operation
43  template <typename T>
44  void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
45      //create the new node
46      qNode<T>* newNode = new_node(newData);
47
48      //inserting into an empty queue
49      if((*frontPtr) == nullptr && (*backPtr) == nullptr){
50          (*frontPtr) = newNode;
51          (*backPtr) = newNode;
52          return;
53      }
54      //inserting an item into a non empty queue
55      (*backPtr)->next = newNode;
56      (*backPtr) = newNode;
57  }
58
59  //dequeue
60  template <typename T>
61  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
62      //create a temporary node
63      qNode<T>* deleteNode = nullptr; // temp variable
64      deleteNode = (*frontPtr);
65
66      //check if the queue has only 1 node
67      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
68          (*frontPtr) = nullptr;
69          (*backPtr) = nullptr;
70          delete deleteNode;
```

```
71          return;
72      }
73      //delete the node
74      (*frontPtr) = (*frontPtr)->next;
75      deleteNode->next = nullptr;
76      delete deleteNode;
77  }
78
79  template <typename T>
80  bool isEmpty(qNode<T>** frontPtr){
81      return (*frontPtr) == nullptr;
82  }
83
84
85  class Printer{
86  private:
87      qNode<Job>* front;
88      qNode<Job>* back;
89
90  public:
91      Printer(){
92          front = nullptr;
93          back = nullptr;
94      }
95
96      void addJob(Job newJob){
97          enqueue(newJob, &front, &back);
98          std::cout << "Job added to the queue: " << newJob.idName << std::endl;
99          std::cout << "Job ID: " << newJob.jobID << std::endl;
100         std::cout << "Pages: " << newJob.pages << std::endl;
101     }
102
103     void processJob(){
104         if(isEmpty(&front)){
105             std::cout << "No jobs in the queue to process." << std::endl;
106             return;
```

```
106             return;
107         }
108         Job jobToProcess = front->data;
109         std::cout << "Processing job: " << jobToProcess.idName << std::endl;
110         std::cout << "Job ID: " << jobToProcess.jobID << std::endl;
111         std::cout << "Pages: " << jobToProcess.pages << std::endl;
112         dequeue(&front, &back);
113     }
114
115     void processAllJobs(){
116         if(isEmpty(&front)){
117             std::cout << "No jobs in the queue to process." << std::endl;
118             return;
119         }
120         while(!isEmpty(&front)){
121             processJob();
122         }
123     }
124
125 };
126
127 #endif
```

Output

```
Adding jobs to the queue...
Job added to the queue: Dirk Sotto
Job ID: 1
Pages: 5
Job added to the queue: John Victoria
Job ID: 2
Pages: 10

Processing jobs in the queue...
Processing job: Dirk Sotto
Job ID: 1
Pages: 5

 Adding more jobs to the queue...
Job added to the queue: Alice Volkswagon
Job ID: 3
Pages: 15
Job added to the queue: King Bob
Job ID: 4
Pages: 20
```

```
Processing remaining jobs in the queue...
Processing job: John Victoria
Job ID: 2
Pages: 10
Processing job: Alice Volkswagon
Job ID: 3
Pages: 15
Processing job: King Bob
Job ID: 4
Pages: 20

Processing remaining jobs in the queue...
No jobs in the queue to process.
```

Code analysis

For the Supplementary Activity, we created an object named myPrinter using the printer class , which implements a queue using a linked list. We first created a queue on the printer by adding two print jobs using the addJob() function. Each job contains a job ID, the name of the user, and the number of pages to be printed. The addJob() function calls the enqueue function, which inserts a new job at the rear of the linked list queue. After adding the first printed job, we tested the processJob() function, which removes and processes the job at the front of the queue using the dequeue() function. We then added two more print jobs to the queue while the printer was busy, which didn't affect the order of the queue. Finally, we called processAllJobs() two times, to repeatedly process each remaining job until the queue was empty, on the last processAllJobs() it displayed the error saying there is no more printer job to be processed.

I chose to use a linked list because the number of users that want to print is not fixed. Unlike arrays where there is a fixed amount, the linked list can dynamically allocate memory based on the jobs that are being added as long as your memory can keep up. Also this avoids overflowing in the queue since there is no fixed max amount on how much you can process in the queue.

Conclusion

Through this activity, I learned how to implement the queue data structure using STL and creating a heart file with arrays and linked lists while understanding the FIFO principle. The procedure part helped me understand and become more familiar on how to do a circular queue and linked listed queues using the operations enqueue, dequeue, front, back, isEmpty, and size. The supplementary activity was challenging but not that much since you just need to implement the basics of dequeue, enqueue and isEmpty, but I think if I am going to use the array it would be a much harder implementation. To conclude, this activity improved my understanding more on how linked lists work. I also think I did okay in this activity because I was able to complete all the required tasks successfully, but on the part of the procedure like circular list, I need more improvements on that because AI guided me on how to do it.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and Code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **ChatGPT was used to help me on how to properly implement the idea on the ILOB2 since the idea of circular queuing is still new to me.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.